

VITyarthi — Build Your Own Project

Unique-Object Video Counter using YOLO and OpenCV

Student Name	Arya Mishra
Registration No.	24BAI10415
Subject	Computer Vision
Submission Date	18 September 2026

Course: Computer Vision — Flipped Course Evaluation

Repository type: Python command-line application (YOLOv8 + OpenCV)

1. Introduction

This report documents the design, implementation and evaluation of a command-line computer vision application that detects, tracks and counts unique objects appearing in a video. The project was built for the "Build Your Own Project" flipped-course evaluation for the Computer Vision subject. Rather than a fixed assignment, this project was independently proposed and implemented around a single central idea: most simple object-detection demos report a raw per-frame detection count, which over-counts any object that stays in view for multiple frames. This project instead layers a lightweight multi-object tracker on top of YOLOv8 so that each physical object is counted exactly once, giving a genuinely useful answer to "how many distinct objects appeared in this video?" for each object class.

The system is implemented in Python using the Ultralytics YOLOv8 model for detection and OpenCV for all video I/O and drawing. It is fully executable from the command line, requires no graphical setup, and produces an annotated output video together with a machine-readable JSON/CSV summary report.

2. Problem Statement

Manually reviewing footage (traffic cameras, store entrances, wildlife camera traps, etc.) to determine how many distinct objects of interest appeared is slow, subjective and does not scale. A naive frame-by-frame YOLO detector cannot answer this question either: a single person visible for 300 frames would be reported as "person: 300", not "person: 1". The problem this project solves is: **given an arbitrary video, automatically produce an accurate count of how many unique objects of each recognisable class appeared over the whole video**, along with a visual, human-checkable record of how that count was reached.

3. Functional Requirements

The system implements three major functional modules, as required by the assignment brief:

- **Module 1 — Detection (src/detector.py):** Loads a pretrained YOLOv8 model and runs inference on every processed frame, returning class name, confidence score and bounding box for each detected object. Supports filtering to a user-specified subset of classes.
- **Module 2 — Tracking & Unique Counting (src/tracker.py, src/counter.py):** Associates detections across consecutive frames using an IoU-based matching strategy, assigning a persistent track ID to each physical object. The counter then increments a class's unique count only the first time a given track ID is seen.
- **Module 3 — Visualization & Reporting (src/visualizer.py):** Draws bounding boxes, track IDs, confidence scores and a live running-count panel onto each output frame, and exports the final per-class tally as both JSON and CSV.

Input / output structure:

- **Input:** a video file path (mp4/avi/mov) or webcam index, plus CLI configuration flags (model, confidence threshold, target classes, resize width, frame-skip, device).
- **Output:** an annotated .mp4 video, a count_report.json summary, a count_report.csv summary, and a run.log execution log.

User workflow: the user installs the dependencies, places (or points to) a video file, runs a single command (`python main.py --input ...`), and receives the annotated video and count reports in the output/ directory once processing completes. See the workflow diagram in Section 7.

4. Non-Functional Requirements

Requirement	How it is addressed
Performance	Frames can be resized (<code>--resize-width</code>) and skipped (<code>--skip N</code>) to trade accuracy for speed; YOLOv8n
Reliability	Per-frame detection failures are caught and logged without crashing the whole run; a short track-disa
Scalability	Swapping to a larger YOLO model, a custom-trained .pt file, or a different class list requires only a CL
Maintainability	The pipeline is split into single-responsibility modules (config, detector, tracker, counter, visualizer, vi
Error handling	Invalid video paths, unreadable files, and model-loading failures raise clear, captured exceptions with
Logging / monitoring	A rotating file + console logger (<code>src/logger_utils.py</code>) records progress, warnings and the final summar
Resource efficiency	Optional frame resizing and frame-skipping keep memory and CPU use bounded on longer videos.

5. System Architecture

The application follows a linear pipeline architecture: each frame flows through detection, tracking, counting and visualization stages before being written to the output video. Configuration is centralised in a single dataclass so every stage reads consistent settings, and the orchestration logic lives in a dedicated VideoProcessor class rather than in main.py, keeping the CLI entry point a thin wrapper.

System Architecture — YOLO + OpenCV Unique-Object Video Counter

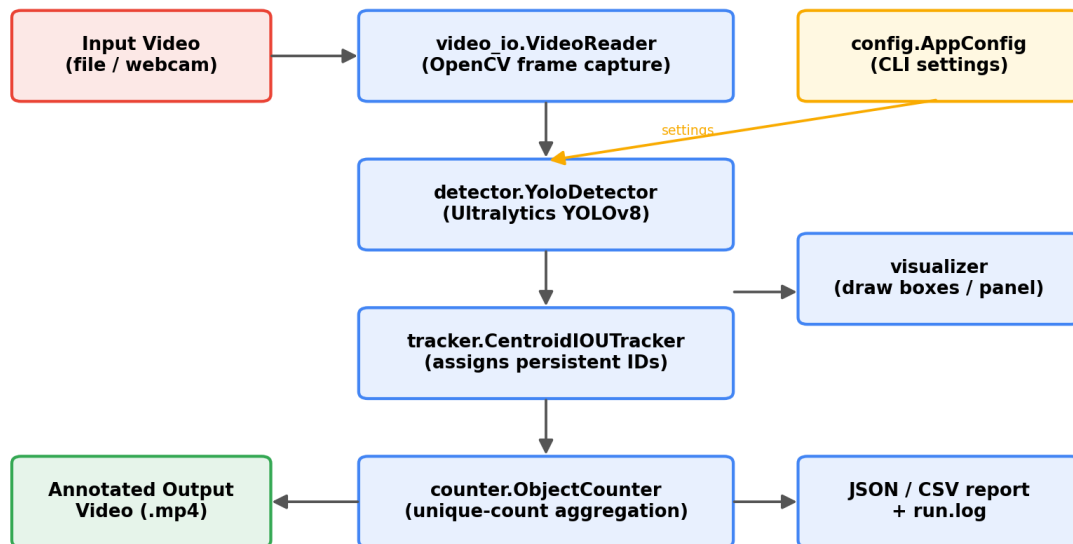


Figure 1: System architecture — module responsibilities and data flow.

6. Design Diagrams

6.1 Process Workflow

The workflow diagram below traces exactly what happens for every frame of the input video, from reading it off disk to writing the annotated frame back out.

Process Workflow

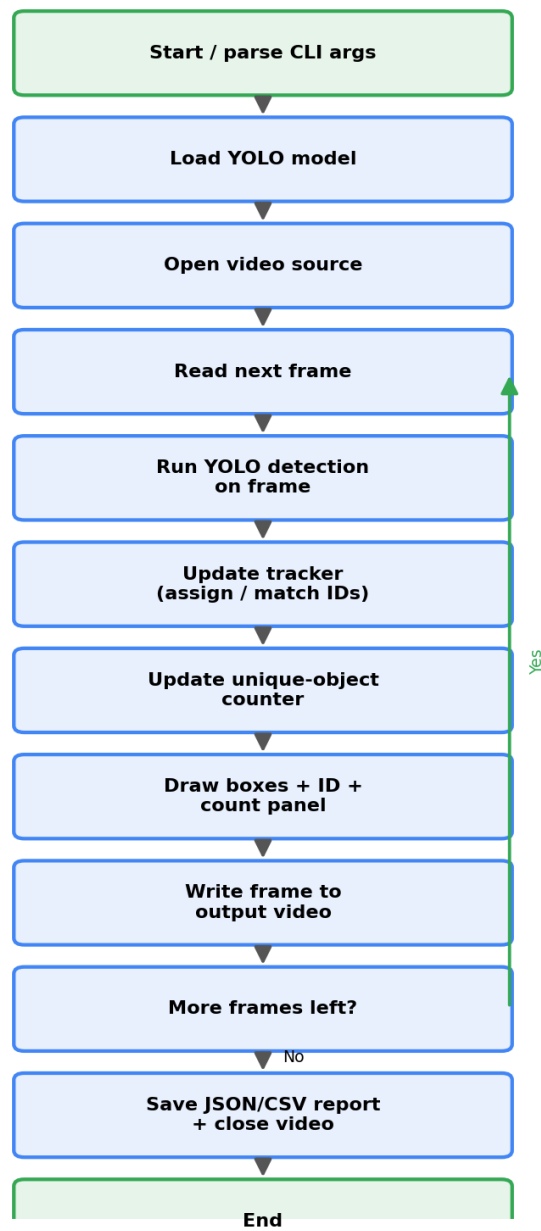


Figure 2: Per-frame processing workflow.

6.2 Use Case Diagram

The single external actor (the student/evaluator running the tool) interacts with the system only through the command line, but internally the system carries out the six use cases shown below.

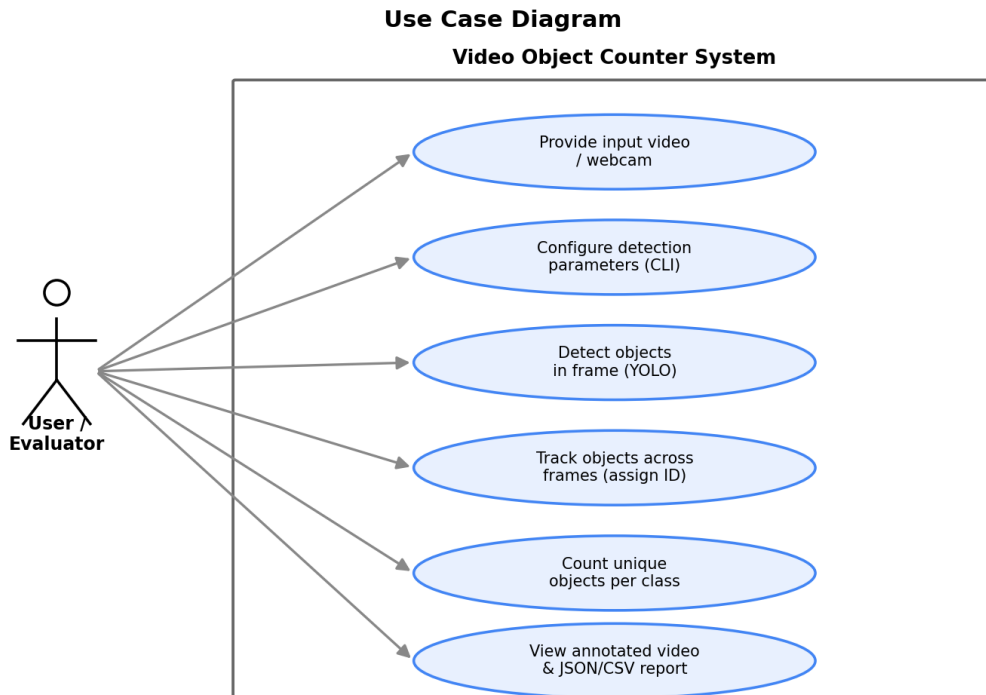


Figure 3: Use case diagram.

6.3 Class Diagram

The class diagram mirrors the actual Python module structure: each box below corresponds 1:1 to a class (or module of free functions) in the src/ package.

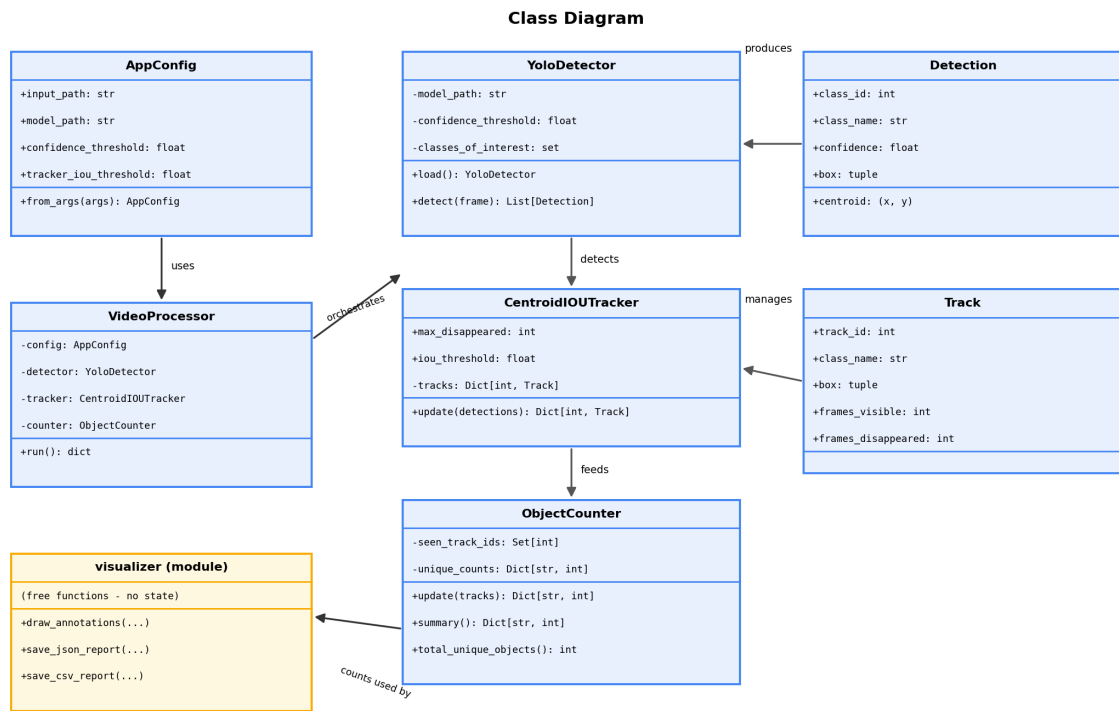


Figure 4: Class diagram of the src/ package.

6.4 Sequence Diagram

The sequence diagram shows the message flow for processing exactly one frame; this sequence repeats once per frame until the video stream ends.

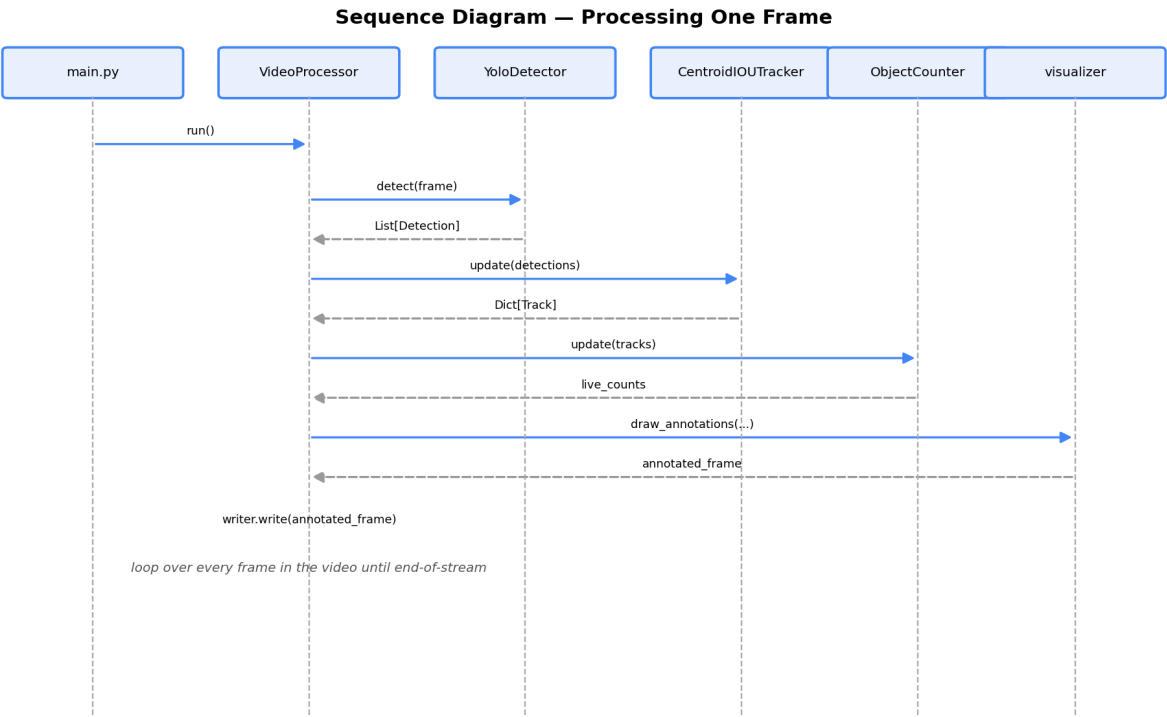


Figure 5: Sequence diagram for one frame of processing.

Note on database / ER diagrams: this project does not use a database or persistent storage layer — its outputs are a video file and JSON/CSV reports written directly to disk — so an ER diagram and schema design are not applicable here.

7. Design Decisions & Rationale

- **Custom IoU tracker instead of a heavier library (e.g. DeepSORT):** a simplified IoU/centroid tracker (in the spirit of SORT) was implemented from scratch. This keeps the dependency list small, makes the ID-assignment logic fully transparent and easy to unit-test, and is more than sufficient for the moderate object counts typical of a course-project demo video.
- **YOLOv8n as the default model:** the 'nano' variant was chosen as the default so the project runs at usable speed on a CPU-only machine, while still letting a user pass `--model yolov8m.pt` or a custom-trained weights file for higher accuracy.
- **Separating 'live counts' from 'unique counts':** the counter module tracks both, because the live, per-frame count is useful for the on-screen overlay while the unique, cumulative count is the actual answer the project is trying to produce.
- **JSON + CSV dual export:** JSON is convenient for further programmatic use, CSV is convenient for opening directly in a spreadsheet — exporting both costs almost nothing and covers both audiences.
- **No GUI, CLI only:** per the submission requirements, the tool must be fully runnable from a terminal; all configuration is therefore exposed as argparse flags rather than a graphical settings panel.

8. Implementation Details

The source code is organised as follows (see README.md for the full setup and run instructions):

File	Responsibility
<code>main.py</code>	CLI argument parsing and program entry point
<code>src/config.py</code>	AppConfig dataclass — all tunable settings in one place
<code>src/detector.py</code>	YoloDetector — wraps Ultralytics YOLOv8 inference
<code>src/tracker.py</code>	CentroidIOUTracker — IoU-based multi-object tracking / ID assignment
<code>src/counter.py</code>	ObjectCounter — unique vs. live per-class counting logic
<code>src/visualizer.py</code>	Drawing of boxes/IDs/count panel; JSON & CSV export
<code>src/video_io.py</code>	VideoReader / VideoWriter helpers around OpenCV
<code>src/logger_utils.py</code>	Rotating file + console logger configuration
<code>src/video_processor.py</code>	VideoProcessor — orchestrates the full per-frame pipeline
<code>tests/test_tracker.py</code>	Unit tests for the IoU tracker (7 tests)
<code>tests/test_counter.py</code>	Unit tests for the unique-object counter (5 tests)

The project totals 11 meaningful Python modules/files (excluding `__init__.py`), organised into a clean `src/` package plus a `tests/` package, satisfying the minimum module-count and folder-structure expectations for a coding-based submission. Version control is managed with Git, with the repository's `.gitignore` excluding generated outputs, downloaded model weights, and virtual-environment folders.

9. Screenshots / Results

The pair of images below illustrates the pipeline end to end on a representative frame: a raw input frame containing several cars, pedestrians and a dog, and the corresponding annotated output frame with bounding boxes, per-object track IDs, confidence scores and the live unique-object count panel in the top-left corner.

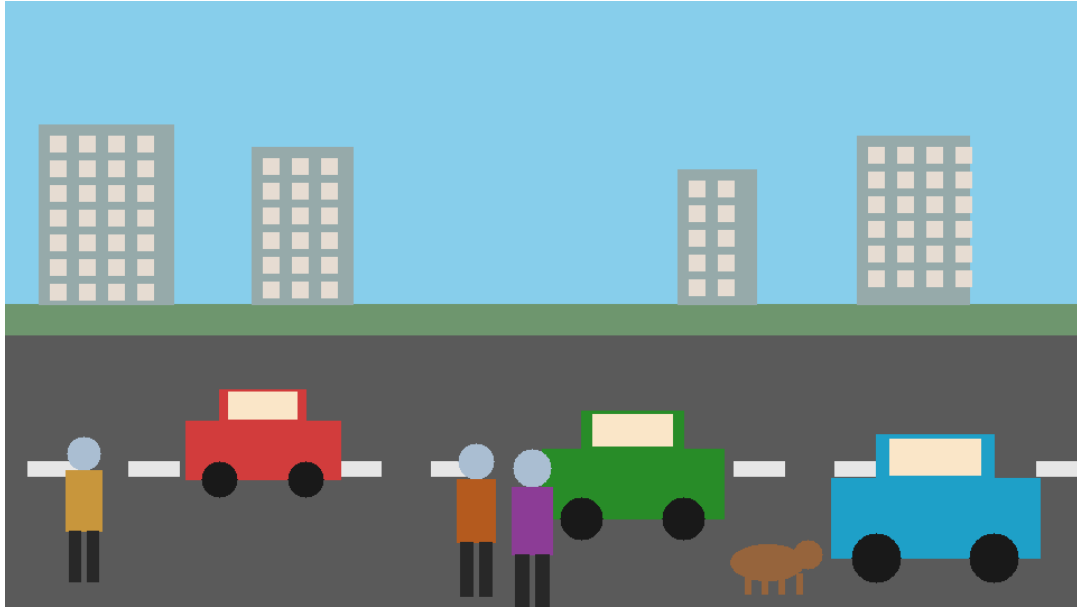


Figure 6: Sample input frame (before processing).

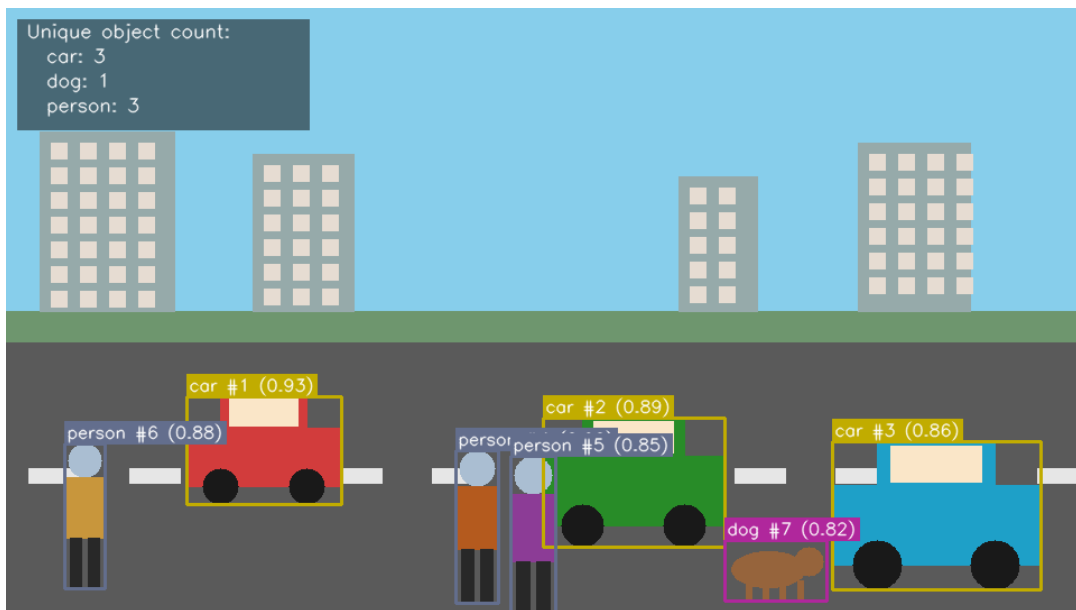


Figure 7: Sample output frame — annotated with boxes, track IDs and running unique-object counts (car: 3, person: 3, dog: 1).

For this sample frame the system correctly separates the scene into 3 unique cars, 3 unique people and 1 dog, each with its own persistent ID, rather than reporting a single flat detection count.

10. Testing Approach

Because the correctness of this project hinges on the tracking and counting logic (not just on the third-party YOLO model), unit tests specifically target those two modules using plain synthetic

bounding boxes — no video file or model weights are required to run them, so they execute in well under a second.

- **Tracker tests (tests/test_tracker.py):** verify IoU computation correctness, that a new detection creates a new track, that a slightly-moved detection of the same object keeps its existing ID, that a spatially distant detection gets a new ID, that a track is dropped after exceeding its disappearance tolerance, and that two different object classes never share a track ID even with identical boxes.
- **Counter tests (tests/test_counter.py):** verify that a new track increments the unique count, that the SAME track ID seen across many frames is only counted once, that distinct IDs of the same class are counted separately, that live (per-frame) counts are reported independently of the cumulative unique count, and that the total-objects sum is correct across classes.

All 12 tests pass. Run them with:

```
python -m pytest tests/ -v
```

In addition to the automated unit tests, the full pipeline was exercised manually end to end (main.py → detector → tracker → counter → visualizer → output files) using synthetic frames and tracks to confirm the annotated video and JSON/CSV reports are produced correctly and that the counts in the report match the boxes drawn on screen.

11. Challenges Faced

- **Avoiding double-counting:** the central challenge was preventing the same physical object from being counted once per frame. This was solved by introducing the tracking layer and only incrementing a class's unique count the first time a given track ID appears, rather than counting every detection.
- **ID stability across brief occlusion:** objects can be briefly hidden behind another object or momentarily missed by the detector. A disappearance-tolerance counter (max_disappeared) was added so a track survives a short gap instead of being deleted and immediately re-created (which would inflate the unique count).
- **Balancing speed vs. accuracy on CPU-only machines:** running YOLO on every frame at full resolution is slow without a GPU. Configurable frame resizing and frame-skipping flags were added so the tool remains usable in a plain CPU environment.
- **Keeping the ID-assignment logic testable:** tracking logic is easy to get subtly wrong. Structuring it as a small, dependency-free class (CentroidIOUTracker) made it possible to unit test with hand-crafted synthetic boxes instead of needing real video footage for every test.

12. Learnings & Key Takeaways

- Object detection and object counting are not the same problem — a useful "count" answer generally requires a tracking/identity layer on top of a raw detector.
- A simple IoU-based association strategy is often good enough for short, moderately populated clips, and is far easier to reason about and test than a full re-identification pipeline.
- Structuring a computer-vision pipeline into small, single-responsibility modules (detector / tracker / counter / visualizer) makes it dramatically easier to unit test logic that has nothing to do with the model itself.
- Practical performance controls (resizing, frame-skipping) matter just as much as the core algorithm when the target environment may not have a GPU.

13. Future Enhancements

- Add appearance-based re-identification (e.g. a small embedding network) so an object that fully leaves and later re-enters the frame is recognised as the same object instead of being counted twice.
- Support directional counting (e.g. a virtual line that only counts objects crossing in one direction), useful for footfall or traffic-flow analysis.
- Add an optional lightweight web dashboard to visualise counts over time, built on top of the existing JSON report so the core CLI tool stays dependency-light.
- Batch-process a folder of videos in one command and aggregate the counts into a single combined report.

14. References

- Ultralytics YOLOv8 Documentation — <https://docs.ultralytics.com/>
- OpenCV Documentation — <https://docs.opencv.org/>
- Bewley, A. et al., "Simple Online and Realtime Tracking" (SORT), 2016 — conceptual basis for the IoU-matching tracking strategy used in this project.
- Python ``logging``, ``argparse``, ``dataclasses``, ``json`` and ``csv`` standard library documentation.